CF 1994E — Wooden Game

Rating: 2000

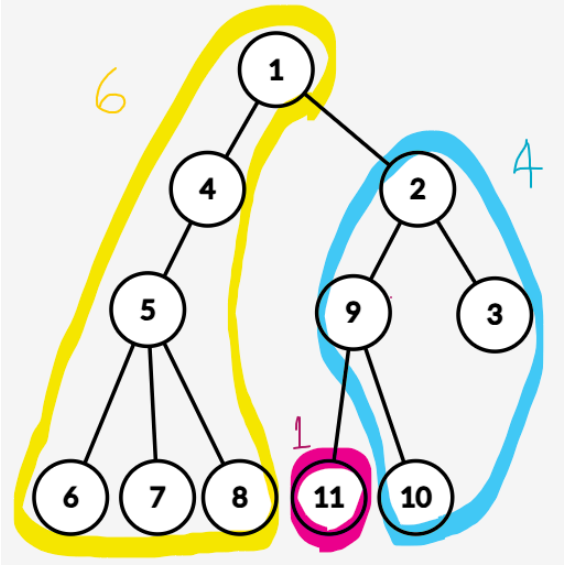
1. Statement

You are given k rooted trees. Every tree is rooted at vertex 1. You want to delete the whole forest by performing the following operation any number of times:

- Select a subtree of some vertex and remove it from the tree.

You want to maximize the bitwise OR of the sizes of the subtrees you remove.

For the following example shown in the image below, we cut the size-1 subtree of vertex 11 first, the size-4 subtree of vertex 2, and then the size-6 subtree of vertex 1. Therefore, the bitwise OR will be $1 \mid 4 \mid 6 = 7$. (This is not the maximum you can reach.)



2. Solution

First, only consider one tree. It can be proven that the maximum bitwise OR can be reached by cutting the whole tree out at once, so if the tree has size n, the maximum result will be n.

> **Proof.** Assume subtrees with sizes $s_1, s_2, \ldots, s_m$ are removed. Consider each bit separately. t_i represents the number of $j \in [1, m]$ such that s_j has the i-th bit on, where i starts from 0. Apparently, $s_1 + s_2 + \ldots + s_m = n$, and
>
> $$s_1 + s_2 + \ldots + s_m = \sum_{i=0}^{20} 2^i t_i = n.$$
>
> The bitwise OR of them will be
>
> $$s_1 \mid s_2 \mid \ldots \mid s_m = \sum_{i=0}^{20} 2^i (t_i > 0).$$
>
> Since $t_i \geq (t_i > 0)$,
>
> $$n = \sum_{i=0}^{20} 2^i t_i \geq \sum_{i=0}^{20} 2^i (t_i > 0),$$

> any separation is either less optimal or equally optimal compared to just deleting the tree all at once and getting the result of n. ■

By deleting some nodes from a size-n tree one by one, any tree the size of $1 \leq i \leq n$ can be achieved, therefore, all bitwise OR values ranging from 1 to n with the 0-th bit on can be obtained. The 0-th bit is not important because leaving it on can only increase the answer, and it does not affect the answer if we just assume all values in $[1, n]$ can be achieved for a tree.

The problem transforms into choosing k numbers, one for each tree, and the number chosen for tree i must be in the range $[1, n_i]$, where n_i is the size of the tree. The goal is to maximize the bitwise OR of the chosen numbers. Well, what if it's not optimal to delete the whole tree at once for this k-tree case? It doesn't matter, since it has been proven that the bitwise OR value for a tree can't exceed n_i, and all values below n_i can be chosen in this solution, covering all possibilities.

Start from the highest bit, 20-th, in this case(19 is also OK). Let i be the current bit, try to find a tree with size larger than or equal to 2^i. If there is no tree satisfying the condition, do nothing. If there is one tree, reduce the size of the tree by 2^i and add 2^i to the final answer. If there are multiple trees, that means all bits from the i-th to the 0-th can be filled up. Keep decreasing the size of one of the trees by one until it become size $2^i - 1$, hence turning all bits below the i-th into ones, and we can break from the loop. It is always optimal to achieve the i-th bit instead of leaving the tree for later bits, because $2^i > 2^{i-1} + 2^{i-2} + ... + 2^0$, that is, no matter how you choose later, it cannot compensate the loss of not achieving the i-th bit now.

3. Code

The structures of the trees are completely useless. Why provide the information? To confuse the problem solvers.

```
#include<bits/stdc++.h>
using namespace std;
int t,k,n[1000005],p;
int main(){
  cin>>t;
  while(t--){
    cin>>k;
    for(int i=1;i≤k;i++){
      cin>>n[i];
      for(int j=1;j<n[i];j++) cin>>p;
    }
    int ans=0;
    for(int i=20;i≥0;i--){
      int flag=0;
      for(int j=1;j≤k;j++)
        if(n[j]≥(1<<i)){
          if(flag){
            ans+=(1<<i)-1;
            flag=-1;
            break;
          }
          flag=1;
          ans+=(1<<i);
          n[j]-=(1<<i);
        }
      if(flag==-1) break;
    }
    cout<<ans<<endl;
```

```
  }
  return 0;
}
```